

1 Roteiro Prático 03 — Atividade Prática Autônoma: Calculadora de Pedidos & Carrinho de Compras

Objetivos

Objetivos da Atividade (Atividade Guiada de 2 Semanas — Calculadora de Pedidos)

Ao final desta atividade prática autônoma, o estudante será capaz de:

- Consolidação prática e aplicação integrada do **Bootstrap 5** e do **jQuery** em uma aplicação web funcional;
- Ler valores de campos de formulário (`<select>`, `<input type="number">`, `<input type="checkbox">`) e convertê-los com segurança para tipos numéricos utilizando `parseFloat()` e `parseInt()`;
- Tratar valores vazios ou não numéricos (NaN) utilizando o operador de fallback `|| 0`;
- Compreender a diferença e aplicar os eventos corretos do navegador: `.on('input')` para digitação em tempo real, `.on('change')` para alteração de seleção/switches e `.on('click')` para botões;
- Realizar o cálculo automático do subtotal, taxa de entrega, descontos por cupom e total geral em tempo real, sem necessidade de recarregar a página;
- Formatar valores monetários para o padrão brasileiro (R\$ X,XX) utilizando a função `.toFixed(2).replace('.', ',')`;
- Persistir e restaurar o rascunho do pedido no navegador utilizando o **LocalStorage** (`setItem`, `getItem`, `JSON.stringify`, `JSON.parse`);
- Organizar o projeto em arquivo externo `app.js` e publicar a aplicação no **GitHub Pages**.

1.1 Introdução e Contextualização da Atividade

Prezados estudantes da disciplina de **Programação para a Internet 2 (ProgWeb 2)**!

Durante as próximas 2 semanas, enquanto o professor estará em afastamento de saúde, vocês realizarão uma atividade prática autônoma guiada para consolidar os conceitos aprendidos sobre **Bootstrap 5**, **jQuery**, **Cálculos Numéricos no Cliente** e **Persistência no LocalStorage**.

O tema proposto é o desenvolvimento da **Lanchonete Express — Calculadora de Pedidos & Carrinho de Compras**. A aplicação permitirá que o cliente escolha seu lanche principal, selecione adicionais opcionais (queijo, bacon, batata frita, bebida), ajuste a quantidade, escolha o tipo de entrega,

aplique um cupom de desconto e veja o **valor total a pagar atualizado instantaneamente na tela**.

1.2 Fundamentação Teórica: Como Ler e Calcular Valores de Campos com jQuery

Para realizar cálculos financeiros em uma página web, é necessário seguir três etapas indispensáveis:

1.2.1 1. Leitura e Conversão de Texto para Número (parseFloat e parseInt)

No HTML, todos os valores obtidos através de `$(seletor).val()` são retornados como **texto (string)**. Se tentarmos somar textos em JavaScript (ex: `"18.00" + "4.00"`), o resultado será uma concatenação (`"18.004.00"`), e não uma soma numérica.

Por isso, devemos converter o valor lido:

- `parseFloat(string)`: Converte o texto para número decimal (usado para preços);
- `parseInt(string)`: Converte o texto para número inteiro (usado para quantidades);
- **Tratamento de NaN (Not a Number)**: Caso o campo esteja vazio, o `parseFloat()` retornará NaN. Para evitar que os cálculos quebrem, usamos a instrução `|| 0`.

Exemplo de leitura segura com fallback zero:

```
const precoLanche = parseFloat($('#select-lanche').val()) || 0;  
const quantidade = parseInt($('#input-qtd').val()) || 1;
```

1.2.2 2. Soma de Múltiplos Checkboxes Marcados com .each()

Quando a aplicação possui múltiplos adicionais (ex: switches de queijo, bacon, batata frita), podemos percorrer apenas os checkboxes que o usuário marcou usando a combinação do seletor `:checked` com o método `.each()`:

Varredura de adicionais selecionados:

```
let somaAdicionais = 0;  
  
$('.check-adicional:checked').each(function() {  
    somaAdicionais += parseFloat($(this).val()) || 0;  
});
```

1.2.3 3. Guia de Eventos do Navegador (input, change e click)

Para proporcionar uma excelente experiência de usuário (UX), a calculadora deve reagir instantaneamente a qualquer alteração realizada na tela. Escolha o evento adequado para cada tipo de elemento:

Guia de Eventos do Navegador em jQuery

Evento jQuery	Componente Indicado e Comportamento
<code>.on('change')</code>	Indicado para select, checkbox e switch. Disparado quando o usuário altera a seleção de um dropdown ou marca/desmarca uma caixa.
<code>.on('input')</code>	Indicado para input type="number" ou text. Disparado instantaneamente a cada caractere digitado ou alterado no campo.
<code>.on('click')</code>	Indicado para botões. Disparado quando o usuário clica nos botões de incremento (+/-), aplicar cupom ou finalizar pedido.

1.2.4 4. Formatação Financeira em Real (R\$)

Para exibir valores numéricos formatados na moeda brasileira (R\$ 25,50), crie uma função auxiliar utilizando `.toFixed(2)` para fixar duas casas decimais e `.replace('.', ',')` para trocar o ponto pela vírgula:

Função de Formatação de Moeda Brasileira:

```
function formatarMoeda(valor) {
    return 'R$ ' + valor.toFixed(2).replace('.', ',');
}

// Atualização da interface gráfica:
$('#total-geral').text(formatarMoeda(totalCalculado));
```

1.3 Dicas de Estruturação Visual & Componentes do Bootstrap 5

Para que a aplicação tenha um visual profissional e limpo, recomendamos a utilização dos seguintes painéis e utilitários do **Bootstrap 5**:

- Grid de 2 Colunas** (`.row` e `.col-lg-7 / .col-lg-5`): Divida a tela em dois painéis principais — a coluna da esquerda para o formulário de montagem do pedido e a coluna da direita para o resumo financeiro;
- Painel de Montagem** (`.card` e `.card-body`): Agrupe as opções do pedido em um cartão elevado com bordas suaves;
- Adicionais com Switches** (`.form-check` e `.form-switch`): Utilize interruptores modernos para os adicionais opcionais (queijo, bacon, fritas), incluindo a etiqueta `.badge` com o preço;
- Input Groups para Quantidade** (`.input-group`): Combine os botões de decremento (-) e incremento (+) com o campo numérico no meio;

5. **Resumo Financeiro em Lista** (`.list-group .list-group-flush`): Exiba a discriminação do subtotal, taxa de entrega e desconto em uma lista estilizada;
6. **Destaque do Total Geral** (`.alert` ou **caixa personalizada**): Destaque o valor total a pagar com tamanho de fonte ampliado e fundo contrastante.

1.4 Roteiro Guiado Passo a Passo (Roteiro 3)

Siga os passos abaixo para construir a aplicação na sua pasta de exercícios (roteiro-3/exercicio-roteiro3/):

Passo 1: Estrutura HTML5 e Inclusão das CDNs

Crie o arquivo `index.html` e inclua as bibliotecas do **Bootstrap 5**, **Bootstrap Icons** e **jQuery 3.7.1** via CDN antes de fechar a tag `</body>`.

```
<!-- Bootstrap 5 CSS -->
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet">
<!-- jQuery CDN -->
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<!-- Arquivo da sua lógica JS -->
<script src="app.js"></script>
```

Passo 2: Formulário de Montagem do Pedido

Crie o menu de seleção do lanche principal atribuindo os preços no atributo `value` de cada `<option>`.

```
<select class="form-select" id="select-lanche">
  <option value="0">-- Selecione um lanche --</option>
  <option value="18.00">X-Burger Clássico - R$ 18,00</option>
  <option value="22.00">X-Salada Especial - R$ 22,00</option>
  <option value="26.00">X-Bacon Supremo - R$ 26,00</option>
</select>
```

Passo 3: Lógica de Cálculo no `app.js`

No arquivo `app.js`, estruture a função `$(document).ready()` e crie a função principal `calcularTotal()`:

```
$(document).ready(function() {

    function calcularTotal() {
        const precoLanche = parseFloat($('#select-lanche').val()) || 0;

        let somaAdicionais = 0;
        $('.check-adicional:checked').each(function() {
            somaAdicionais += parseFloat($(this).val()) || 0;
        });

        let qtd = parseInt($('#input-qtd').val()) || 1;
        const subtotal = (precoLanche + somaAdicionais) * qtd;
        const taxaEntrega = parseFloat($('#select-entrega').val()) || 0;

        const totalGeral = subtotal + taxaEntrega;

        $('#total-geral').text('R$ ' + totalGeral.toFixed(2).replace('.', ','));
    }

    // Escutadores em tempo real
    $('#select-lanche, #select-entrega, .check-adicional').on('change', calcularTotal);
    $('#input-qtd').on('input change', calcularTotal);

});
```

Passo 4: Persistência no LocalStorage

Implemente o salvamento automático do rascunho do pedido no `localStorage` ao clicar em "Finalizar Pedido" e a restauração automática ao pressionar F5:

```
// Salvar no LocalStorage
$('#btn-finalizar').on('click', function() {
  const pedido = {
    lanche: $('#select-lanche').val(),
    qtd: $('#input-qtd').val(),
    entrega: $('#select-entrega').val()
  };
  localStorage.setItem('rascunho_pedido', JSON.stringify(pedido));
  alert('Pedido salvo no navegador!');
});
```

1.5 Checklist de Conclusão e Orientações de Entrega

Antes de enviar sua atividade, verifique se você concluiu todos os itens da lista abaixo:

- ☐ A página carrega corretamente sem erros no console do navegador (F12);
- ☐ O layout é responsivo e utiliza os cartões e formulários do Bootstrap 5;
- ☐ O cálculo do valor total reage em tempo real a qualquer mudança nos selects, checkboxes ou quantidade;
- ☐ Os valores numéricos são devidamente formatados em Reais (R\$ X,XX);
- ☐ O rascunho do pedido é salvo e restaurado via localStorage;
- ☐ O código foi commitado no repositório Git pessoal e publicado via **GitHub Pages**.